

1. 题目深度解构

题意重述

给定数组 $a_1, \dots, a_n$ 。

一次操作选择一个**长度为偶数**的区间 $[l, r]$ ，然后：

$$a_l + = 1, \quad a_{l+1} - = 1, \quad a_{l+2} + = 1, \quad a_{l+3} - = 1, \dots$$

问最少多少次操作可以使所有数相等。

约束分析

$$\sum n \leq 10^6, \quad a_i \leq 10^{12}$$

因此需要 $O(n)$ 或 $O(n \log n)$ ，最好是 $O(n)$ 。

数值上：

$$na_i \leq 10^{18}$$

必须使用 `long long`。

题目类型判定

这是典型的：

- Ad-hoc / 思维题
- 前缀和不变量
- 区间覆盖最小次数
- 贪心

核心不是数据结构，而是发现操作对**前缀和**的影响。

2. 思维路径推导

第一步：总和不变

一次操作中，区间长度为偶数，内部加一和减一的数量相等。

所以数组总和不变。

如果最终所有数都变成 x ，那么：

$$nx = \sum_{i=1}^n a_i$$

因此必须满足：

$$\sum a_i \equiv 0 \pmod n$$

令：

$$x = \frac{\sum a_i}{n}$$

最终目标唯一确定。

但这还不够。

第二步：为什么平均数存在仍然可能无解？

看前面反例：

$$[3, 1]$$

平均数是 2。

但操作 $[1, 2]$ 会让数组变成：

$$[4, 0]$$

无法降低第一个元素。

这说明操作有方向性，不能简单地看成“某些位置加减一”。

于是我们换一个角度：看前缀和。

第三步：关键观察：操作只会增加某些前缀和

设：

$$S_k = a_1 + a_2 + \cdots + a_k$$

考虑一次操作 $[l, r]$ 。

它对前缀和 S_k 的影响：

- 如果 $k < l$ ，没影响。
- 如果 $k \geq r$ ，整个操作区间都被包含，区间总变化为 0，也没影响。
- 如果 $l \leq k < r$ ，前缀只截取了操作区间的一部分。

这一部分是：

$$+1, -1, +1, -1, \dots$$

如果截取长度是奇数，净变化为 +1。

如果截取长度是偶数，净变化为 0。

因此一次操作只会让某些前缀和增加 1，绝不会让任何前缀和减少。

更具体地说，操作 $[l, r]$ 会让：

$$S_l, S_{l+2}, S_{l+4}, \dots, S_{r-1}$$

这些前缀和各增加 1。

第四步：得到必要条件

最终所有数都等于 x ，所以最终前缀和应该是：

$$S'_k = kx$$

而操作过程中每个前缀和只能增加，不能减少。

所以必须满足：

$$S_k \leq kx$$

也就是：

$$kx - S_k \geq 0$$

对所有：

$$1 \leq k < n$$

都必须成立。

如果某个前缀和已经超过最终目标前缀和，那么无论怎么操作都降不下来，直接无解。

第五步：把问题变成“削平柱状图”

定义前缀差值：

$$d_k = kx - S_k$$

它表示第 k 个前缀还需要增加多少次。

我们希望通过操作把所有 d_k 降到 0。

一次操作 $[l, r]$ 会让：

$$S_l, S_{l+2}, \dots, S_{r-1}$$

增加 1，所以会让：

$$d_l, d_{l+2}, \dots, d_{r-1}$$

减少 1。

也就是说：

一次操作可以在同奇偶性的一段连续位置上，把 d 全部减一。

于是奇数位置的 d 和偶数位置的 d 完全独立。

例如：

$$d_1, d_3, d_5, \dots$$

是一组；

$$d_2, d_4, d_6, \dots$$

是另一组。

对每一组，我们的问题变成：

给定一排非负高度，每次可以选择一个连续区间全部减一，最少多少次把它们全部变成 0。

这是经典的“区间覆盖 / 柱状图削平”问题。

答案是：

$$h_1 + \sum_{i=2}^m \max(0, h_i - h_{i-1})$$

直觉是：每当高度比前一个位置更高时，多出来的部分必须新开操作区间。

算法确立

扫描数组，维护前缀和。

对于每个 $k < n$ ，计算：

$$d_k = kx - S_k$$

如果 $d_k < 0$ ，无解。

否则按照 k 的奇偶性，维护上一项高度 `last[parity]`。

如果：

$$d_k > \text{last}[\text{parity}]$$

说明需要新开：

$$d_k - \text{last}[\text{parity}]$$

个操作。

累加进答案。

3. Trick 与陷阱

Trick 点拨

核心 Trick 是：

不看单点变化，而看前缀和变化。

原数组中的交替 $+1, -1, +1, -1$ ，在前缀和视角下变成了：

某些同奇偶位置的前缀和统一增加 1。

这一步直接把题目变成了两个独立的区间覆盖问题。

易错点

1. 平均数存在不代表有解

反例：

$$[3, 1]$$

平均数是 2，但无解。

2. 操作不可逆

不能认为区间 $[l, r]$ 可以自由选择正负方向。

3. 必须检查所有前缀

条件是：

$$S_k \leq kx$$

对所有 k 成立。

4. 使用 `long long`

因为：

$$n \cdot a_i \leq 10^{18}$$

5. $n = 1$

不需要操作，答案为 0。

4. 代码实现 C++23 / jiangly Style

```
#include <bits/stdc++.h>

using i64 = long long;

int main() {
    std::cin.tie(nullptr) -> sync_with_stdio(false);

    int T;
```

```

std::cin >> T;

while (T--) {
    int n;
    std::cin >> n;

    std::vector<i64> a(n + 1);
    i64 sum = 0;

    for (int i = 1; i <= n; i++) {
        std::cin >> a[i];
        sum += a[i];
    }

    if (sum % n != 0) {
        std::cout << -1 << "\n";
        continue;
    }

    i64 x = sum / n;
    i64 pref = 0;
    i64 ans = 0;
    i64 last[2] = {};

    bool ok = true;

    for (int i = 1; i <= n; i++) {
        pref += a[i];
        i64 d = 1LL * i * x - pref;

        if (i < n) {
            if (d < 0) {
                ok = false;
            }

            int p = i & 1;

            if (ok && d > last[p]) {
                ans += d - last[p];
            }

            if (ok) {
                last[p] = d;
            }
        }
    }

    if (!ok) {
        std::cout << -1 << "\n";
    } else {
        std::cout << ans << "\n";
    }
}

```

```
    return 0;  
}
```

5. 总结与启示

难度评分

预估 Codeforces Rating: **1800 左右**。

核心启示

对交替加减操作，直接看元素很乱；转成前缀和后，操作变成了“只增加某些前缀”，结构立刻清晰。

同类习题推荐

1. **CF 448C - Painting Fence**
经典区间操作削平柱状图。
2. **CF 1373C - Pluses and Minuses**
前缀和视角处理操作代价。
3. **AtCoder ABC 136 / ARC 系列中的前缀构造类题目**
适合练习“局部操作转前缀约束”的思维。